



TermProject[•]

2023 Data Base System

Team C

202132826 배지희

202135749 김지민

202135821 이희재

202135838 조선민

Contents[•]

Contents 01

First DB
schema design

Contents 02

Normalization

Contents 03

Final DB schema

Contents 04

SQL DDL

Contents 05

SQL DML

Contents 06

Demonstration



USER, ORGANIZATION, GROU, PHOTO
POST, POST_TAG, POST_LIKE, SHARE
COMMENT, COMMENT_TAG, COMMENT_LIKE

FOLLOWER, FOLLOWING, FOLLOW,
BELONG, WRITE, HAVE

Normalization[•]

1NF

IF THERE ARE TWO PHONE_NUMBERS PER PERSON,
TWO PIECES OF DATA ARE CONTAINED IN ONE CELL



1NF

THIS MUST BE DIVIDED BECAUSE IT VIOLATES 1NF,
WHICH REQUIRES THE COLUMNS OF THE TABLE TO HAVE ATOMIC VALUES



1NF

SO, INCREASE CELLS FROM PHONE_NUMBER → PHONE_NUM1, PHONE_NUM2



Normalization[•]

2NF

USER_ID UNIQUELY IDENTIFIES EACH ATTRIBUTE OF THE RELATION
THEREFORE, 2NF IS SATISFIED



3NF

{USER_ID→BIRTHDAY, BIRTHDAY→AGE, USER_ID→AGE} TRANSITION FUNCTION DEPENDENCY EXISTS,
SO THE TABLE MUST BE SEPARATED INTO {USE_ID, BIRTHDAY+ETC.}/{USER_ID, AGE}



3NF

ALL OTHER PROPERTIES ARE DETERMINED ONLY THROUGH USER_ID,
WHICH IS THE PRIMARY KEY

Normalization

FIRST USER TABLE

```
create table user(  
  user_ID varchar(50),  
  password varchar(50),  
  user_name varchar(50),  
  phone_number varchar(20),  
  birthday date,  
  address varchar(50),  
  age int,  
  follower int,  
  following int,  
  primary key(user_ID));
```



USER TABLE AFTER 1NF

```
create table user(  
  user_ID varchar(50),  
  password varchar(50),  
  user_name varchar(50),  
  phone_number1 varchar(20),  
  phone_number2 varchar(20),  
  birthday date,  
  address varchar(50),  
  age int,  
  follower int,  
  following int,  
  primary key(user_ID));
```



FINAL USER TABLE

```
create table user(  
  user_ID varchar(50),  
  password varchar(50),  
  user_name varchar(50),  
  phone_number1 varchar(20),  
  phone_number2 varchar(20),  
  birthday date,  
  age int,  
  address varchar(50),  
  follower int,  
  following int,  
  primary key(user_ID));
```

```
create table birth(  
  user_ID varchar(50),  
  age int,  
  primary key(birthday));
```

Final DB schema •

- USER(**USER_ID**, PASSWORD, USER_NAME, PHONE_NUMBER1, PHONE_NUMBER2, BIRTHDAY, ADDRESS, FOLLOWER, FOLLOWING)
- BIRTH(**USER_ID**, AGE) FOREIGNKEY- **USER_ID**(REFERENCE USER TABLE)
- POST(**POST_ID**, POST_DATE, BODY_TEXT, AUTHOR_ID) FOREIGNKEY- **AUTHOR_ID**(REFERENCE USER TABLE)
- POST_LIKE(**USER_ID**, **POST_ID**)
- FOLLOWER(**USER_ID**, **FOLLOWER_ID**) FOREIGNKEY- **USER_ID/FOLLOWER_ID**(REFERENCE USER TABLE)
- FOLLOWER(**USER_ID**, **FOLLOWING_ID**) FOREIGNKEY- **USER_ID/FOLLOWING_ID**(REFERENCE USER TABLE)
- COMMENT(**COMMENT_ID**, USER_ID, COMMENT_CONTENT, COMMENT_DATE)
FOREIGNKEY- **USER_ID**(REFERENCE USER TABLE)/**POST_ID**(REFERENCE POST_ TABLE)
- COMMENT_LIKE(**USER_ID**, **COMMENT_ID**)
FOREIGNKEY- **USER_ID**(REFERENCE USER TABLE)/**COMMENT_ID**(REFERENCE COMMENT_ TABLE)
- COMMENT_TAG(**COMMENT_ID**, **USER_ID**, **TAGGED_ID**)
FOREIGNKEY- USER_ID/TAGGED_ID (REFERENCE USER TABLE)/**COMMENT_ID**(REFERENCE COMMENT_ TABLE)

SQL DDL

-  create table user(
user_ID varchar(50),
password varchar(50),
user_name varchar(50),
phone_number varchar(20),
birthday date,
address varchar(50),
follower int,
following int,
phone_number2 varchar(50),
primary key(user_ID));
-  create table post_likes(
user_ID varchar(50),
post_ID varchar(50),
primary key(user_ID, post_ID)
);

-  create table birth (
user_ID varchar(50),
age int,
primary key (user_ID),
foreign key (user_ID) references user(user_ID)
);
-  create table post(
post_ID varchar(50),
post_date datetime,
body_text varchar(500),
author_ID varchar(50),
primary key (post_ID),
foreign key (author_ID) references user(user_ID)
);

SQL DDL

- ```
create table comment(
 comment_ID varchar(50),
 user_ID varchar(50),
 comment_content varchar(250),
 comment_date DATETIME,
 post_ID varchar(50),

 primary key (comment_ID),
 foreign key (user_ID) references user(user_ID),
 foreign key (post_ID) references post(post_ID)
);
```
- ```
create table comment_tag(  
  comment_ID varchar(50),  
  user_ID varchar(50),  
  tagged_ID varchar(50),  
  primary key (user_ID, tagged_ID, comment_ID),  
  foreign key (user_ID) references user(user_ID),  
  foreign key (tagged_ID) references user(user_ID),  
  foreign key (comment_ID) references comment(comment_ID)  
);
```

- ```
create table comment_likes(
 user_ID varchar(50),
 comment_ID varchar(50),

 primary key (user_ID, comment_ID),
 foreign key (comment_ID) references comment(comment_ID),
 foreign key (user_ID) references user(user_ID)
);
```
- ```
create table following(  
  user_ID varchar(50),  
  following_ID varchar(50),  
  primary key (user_ID, following_ID),  
  foreign key (user_ID) references user(user_ID),  
  foreign key (following_ID) references user(user_ID)  
);
```
- ```
create table follower(
 user_ID varchar(50),
 follower_ID varchar(50),
 primary key (user_ID, follower_ID),
 foreign key (user_ID) references user(user_ID),
 foreign key (follower_ID) references user(user_ID)
);
```

# SQL DML

- `insert into user values`

```
("11111","eeeeee","Timmy","010-0104-1234","1999-01-30","Seoul, Gangnam-gu, Yeoksam-dong 123",5,1,null),
("12222","aaaaa","Jimmy","010-1234-1234","1996-02-28","Incheon, Namdong-gu, Ganseok-dong 456",3,2,null),
("13333","abbbb","sunny","010-0293-3827","1967-03-11","Daejeon, Yuseong-gu, Bongmyeong-dong 789",2,3,null),
("14444","acccc","carry","010-4926-6632","2003-04-01","Busan, Haeundae-gu, Jwadong 101",3,3,null),
("55551","adddd","jessie","010-3847-2047","2002-05-05","Gyeongsangbuk-do, Pohang-si, Buk-gu",0,0,null),
("66661","aeeee","sophia","010-0854-2345","2012-06-06","Chungcheongnam-do, Cheonan-si, Seo-bukgu",1,1,null),
("77771","bbbbb","tom","010-9262-8865","2011-07-17","Gwangju, Seo-gu, Nongseong-dong 890",2,5,null),
("88882","abcab","william","010-2345-9854","2009-08-28","Daegu, Suseong-gu, Manchon-dong 112",4,3,null),
("99992","ddddd","willson","010-1235-5673","1986-09-16","Gangwon-do, Chuncheon-si, Namsan-myeon 345",3,2,null),
("10001","eeeeee","son","010-3097-1234","1968-11-13","Jeollabuk-do, Jeonju-si, Deokjin-gu",2,1,null),

("1111","aaab","Aaron","010-4512-1052","2002-09-07","Gyeonggi-do, Gwacheon-si, Munwon-dong 10",2,2,null),
("2222","bbbc","Abigail","010-6622-2620","2002-04-15","Gyeonggi-do, Anyang-si, Dongan-gu",1,3,null),
("3333","cccd","Brandon","010-5556-2623","2000-11-07","Gyeonggi-do, Gunpo-si, Sanbon-dong 20",1,1,null),
("4444","ddde","Bradley","010-4442-4000","2001-06-06","Gyeonggi-do, Hwaseong-si, Banwol-dong 30",0,1,null),
("5555","eeef","Bruce","010-8282-5640","2000-05-05","Seoul, Seongdong-gu, Majanf-dong 40",3,1,null),
("6666","ffffg","Carter","010-0352-2950","2001-08-15","Seoul, Gangnam-gu, Daechi-dong 50",2,1,null),
("7777","gggh","Elliott","010-0163-3500","1999-10-17","Seoul, Gangdong-gu, Cheonho-dong 60",1,0,null),
("8888","hhhi","Felix","010-7004-4660","1998-07-06","Busan, Haeundae-gu, Banson-dong 70",2,2,null),
("9999","iiij","Hudson","010-0603-3058","1996-12-30","Daegu, Suseong-gu, Beomeo-dong 80",1,3,null),
("1010","jjjk","Heath","010-0894-4700","1997-04-16","Daejeon, Yuseong-gu, Jinjam-dong 90",2,1,null),

("111","aaazx","aaron","010-6444-7585","2002-02-14","Seoul, Mapo-gu, Sungsan-dong 37",3,2,null),
("222","bbbxz","adam","010-2710-0579","1970-07-05","Gyeongsangbuk-do, Gumi-si, Jisan-dong 588",4,1,null),
("333","ccczx","bryce","010-4809-0732","2008-09-21","Gwangju, Dong-gu, Duam-dong 323",1,1,null),
("444","dddzx","brian","010-5537-2581","1998-11-29","Busan, Saga-gu, Gaejong-dong 99",2,1,null),
("555","eeezx","dylan","010-4041-3769","1981-10-02","Daegu, Dong-gu, Dongho-dong 370",4,2,null),
("666","fffzx","elliott","010-4856-8396","2003-08-07","Seoul, Songbuk-gu, Majang-dong 241",1,4,null),
("777","gggzx","finn","010-9501-5003","1977-02-21","Incheon, Michuhoel-gu, Gyengwon-dong 31",1,3,null),
("888","hhhzx","henry","010-3026-0177","2011-01-10","Seoul, Songpa-gu, bubwon-ro 29",2,3,null),
("999","iizx","ishmael","010-5523-6080","2004-08-09","Gyeonggi, Buchon-si, Bugwang-ro 64",1,2,null),
("101","jjjzx","kaden","010-9733-3448","1992-04-15","Seoul, Seocho-gu, Narutae-ro 9",1,1,null),
```

# SQL DML

# Login -> MainPage: 가입된 유저 정보랑 일치하면 메인페이지로 넘어감

- `SELECT password FROM user WHERE user_id = "" + ID + "";`

#PasswordChange: 유저의 등록된 비밀번호가 맞는지 확인

- `SELECT password FROM user WHERE user_ID = myID;`

#PasswordChange: 비밀번호 변경(업데이트)

- `UPDATE user SET password = ? WHERE user_ID = myID;`

#SignUp: User table에 data 저장

- `INSERT INTO user (user_ID, password, user_name, phone_number, birthday, address, follower, following, phone_number2)  
VALUES (?, ?, ?, ?, ?, ?, 0, 0, null);`

#MainPage -> following user's posts: 메인페이지에서 팔로잉한 사람들이 올린 포스트 보기

- `SELECT author_ID, post_date, body_text, post_ID FROM post  
WHERE author_ID = myID OR author_ID IN (SELECT following_ID FROM following  
WHERE user_ID = myID) ORDER BY post_date DESC;`

#UserPage -> follower list: 유저페이지에서 팔로워 리스트 보기

- `SELECT follower_ID FROM follower WHERE user_id = "" + ID + "";`

#UserPage -> following list: 유저페이지에서 팔로잉 리스트 보기

- `SELECT following_ID FROM following WHERE user_id = "" + ID + "";`

# SQL DML

#FollowUser: 다른 사람 팔로우하기(팔로잉 수 증가 및 팔로잉 정보 추가)

- `UPDATE user SET following = following + 1 WHERE user_ID = myID;`
- `INSERT INTO following VALUES (myID, followingID);`

#FollowUser: 다른 사람 팔로우하기(팔로워 수 증가 및 팔로워 정보 추가)

- `UPDATE user SET follower = follower + 1 WHERE user_ID = myID;`
- `INSERT INTO follower VALUES (myID, followerID);`

#Post: 포스트 전문 보기

- `SELECT * FROM post WHERE post_ID = "" + ID + "";`

#PostEditor: 포스트 쓰기

- `INSERT INTO post (post_ID, post_date, body_text, author_ID) VALUES (?, ?, ?, ?);`

#CommentGUI: comment 최신 순으로 보기

- `select * from comment left join (select comment_ID, count(user_ID) as likes from comment_likes group by comment_ID) as c_likes using(comment_ID) WHERE post_ID = ? order by comment_date desc;`

#CommentEditor: 태그 없는 comment 쓰기

- `INSERT INTO comment (comment_ID, user_ID, comment_content, comment_date, post_ID) VALUES (?, ?, ?, ?, ?);`

#CommentEditor: 태그 있는 comment 쓰기

- `INSERT INTO comment_tag (comment_ID, user_ID, tagged_ID) VALUES (?, ?, ?);`

#LikePost: 포스트에 좋아요 누르기

- `INSERT INTO post_likes VALUES (myID, postID);`

#LikeComment: comment에 좋아요 누르기

- `INSERT INTO comment_likes VALUES (myID, commentID);`

**Demonstration** •

2023 Data Base System

# Thank you

TEAM C

202132826 배지희 - DB DESIGN/FRONT-END

202135749 김지민 - DB DESIGN/FRONT-END

202135821 이희재 - DB DESIGN/BACK-END

202135838 조선민 - DB DESIGN/BACK-END

GITHUB

[HTTPS://GITHUB.COM/HEEJAE-L/FACEBOOK](https://github.com/HEEJAE-L/FACEBOOK)

